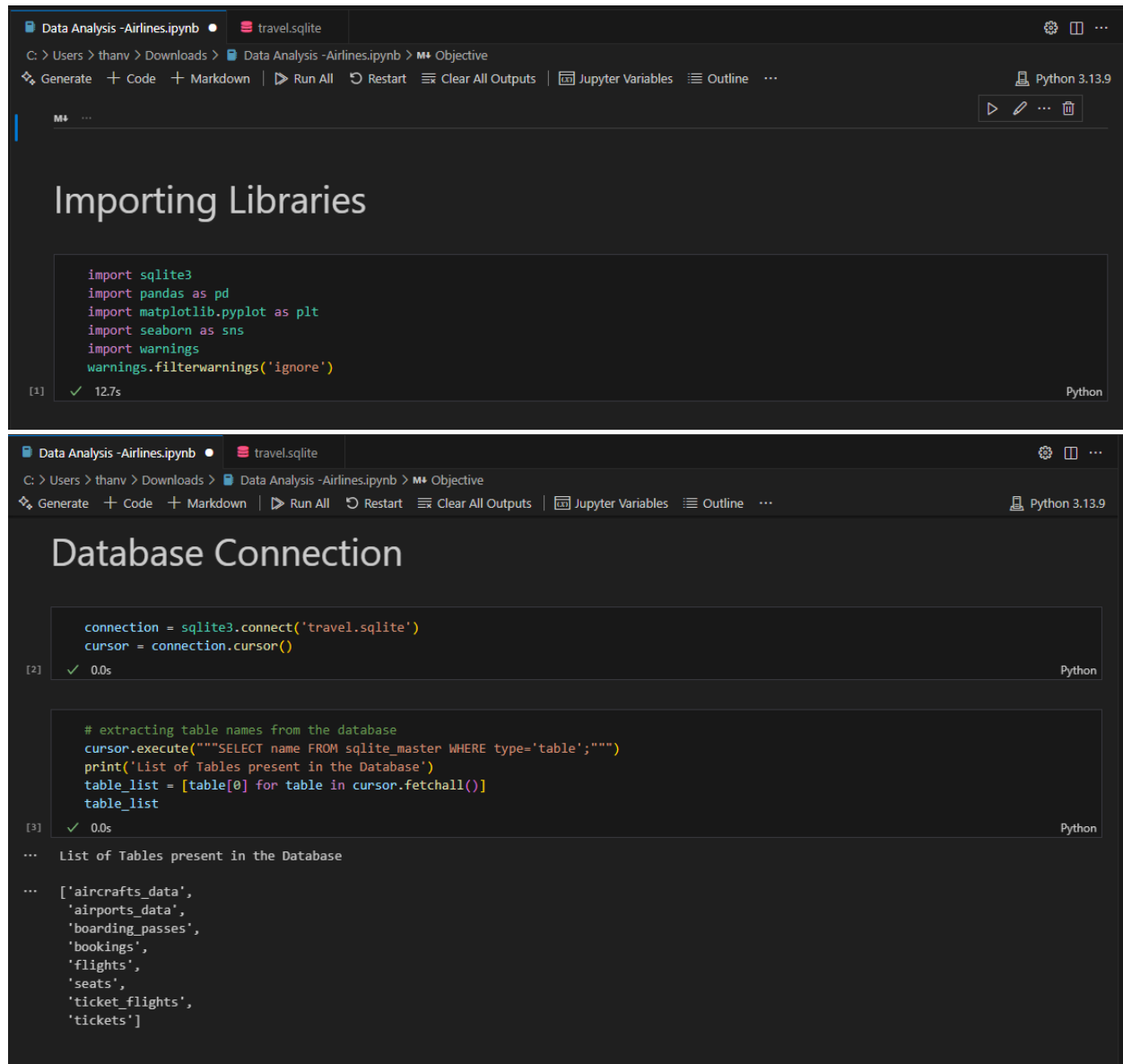


AIRLINES-DATA ANALYSIS



The screenshot displays a Jupyter Notebook interface with two code cells. The first cell, titled "Importing Libraries", contains the following code:

```
import sqlite3
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import warnings
warnings.filterwarnings('ignore')
```

The second cell, titled "Database Connection", contains the following code:

```
connection = sqlite3.connect('travel.sqlite')
cursor = connection.cursor()

# extracting table names from the database
cursor.execute("""SELECT name FROM sqlite_master WHERE type='table';""")
print('List of Tables present in the Database')
table_list = [table[0] for table in cursor.fetchall()]
table_list
```

The output of the second cell is displayed below the code:

```
... List of Tables present in the Database
... ['aircrafts_data',
     'airports_data',
     'boarding_passes',
     'bookings',
     'flights',
     'seats',
     'ticket_flights',
     'tickets']
```

Data Analysis -Airlines.ipynbtravel.sqlite

C: > Users > thanv > Downloads > Data Analysis -Airlines.ipynb > Objective

GenerateCodeMarkdownRun AllRestartClear All OutputsJupyter VariablesOutlinePython 3.13.9

Data Exploration

```
aircrafts_data = pd.read_sql_query(f"""SELECT * FROM aircrafts_data""", connection)
aircrafts_data.head()
```

[4] ✓ 0.0s Python

	aircraft_code		model	range
0	773	{ "en": "Boeing 777-300", "ru": "Боинг 777-300" }		11100
1	763	{ "en": "Boeing 767-300", "ru": "Боинг 767-300" }		7900
2	SU9	{ "en": "Sukhoi Superjet-100", "ru": "Сухой Сун..." }		3000
3	320	{ "en": "Airbus A320-200", "ru": "Аэробус A320-..." }		5700
4	321	{ "en": "Airbus A321-200", "ru": "Аэробус A321-..." }		5600

```
airports_data = pd.read_sql_query(f"""SELECT * FROM airports_data""", connection)
airports_data.head()
```

[5] ✓ 0.0s Python

	airport_code	airport_name	city	coordinates	timezone
0	YKS	{ "en": "Yakutsk Airport", "ru": "Якутск" }	{ "en": "Yakutsk", "ru": "Якутск" }	(129.77099609375,62.0932998657226562)	Asia/Yakutsk
1	MJZ	{ "en": "Mirny Airport", "ru": "Мирный" }	{ "en": "Mirnyj", "ru": "Мирный" }	(114.03900146484375,62.534698486328125)	Asia/Yakutsk

Data Analysis -Airlines.ipynbtravel.sqlite

C: > Users > thanv > Downloads > Data Analysis -Airlines.ipynb > Objective

GenerateCodeMarkdownRun AllRestartClear All OutputsJupyter VariablesOutlinePython 3.13.9

[5] ✓ 0.0s Python

	airport_code	airport_name	city	coordinates	timezone
0	YKS	{ "en": "Yakutsk Airport", "ru": "Якутск" }	{ "en": "Yakutsk", "ru": "Якутск" }	(129.77099609375,62.0932998657226562)	Asia/Yakutsk
1	MJZ	{ "en": "Mirny Airport", "ru": "Мирный" }	{ "en": "Mirnyj", "ru": "Мирный" }	(114.03900146484375,62.534698486328125)	Asia/Yakutsk
2	KHV	{ "en": "Khabarovsk-Novy Airport", "ru": "Хабар..." }	{ "en": "Khabarovsk", "ru": "Хабаровск" }	(135.18800354004,48.5279998779300001)	Asia/Vladivostok
3	PKC	{ "en": "Yelizovo Airport", "ru": "Елизово" }	{ "en": "Petropavlovsk", "ru": "Петропавловск-К..." }	(158.453994750976562,53.1679000854492188)	Asia/Kamchatka
4	UUS	{ "en": "Yuzhno-Sakhalinsk Airport", "ru": "Хом..." }	{ "en": "Yuzhno-Sakhalinsk", "ru": "Южно-Сахалин..." }	(142.718002319335938,46.8886985778808594)	Asia/Sakhalin

```
boarding_passes = pd.read_sql_query(f"""SELECT * FROM boarding_passes""", connection)
boarding_passes.head()
```

[6] ✓ 1.0s Python

	ticket_no	flight_id	boarding_no	seat_no
0	0005435212351	30625	1	2D
1	0005435212386	30625	2	3G
2	0005435212381	30625	3	4H
3	0005432211370	30625	4	5D
4	0005435212357	30625	5	11A

Data Analysis - Airlines.ipynb • travel.sqlite

C: > Users > thanv > Downloads > Data Analysis - Airlines.ipynb > Objective

Generate + Code + Markdown Run All Restart Clear All Outputs Jupyter Variables Outline Python 3.13.9

```
bookings = pd.read_sql_query(f"""SELECT * FROM bookings """, connection)
bookings.head()
```

[7] ✓ 0.4s Python

	book_ref	book_date	total_amount
0	00000F	2017-07-05 03:12:00+03	265700
1	000012	2017-07-14 09:02:00+03	37900
2	000068	2017-08-15 14:27:00+03	18100
3	000181	2017-08-10 13:28:00+03	131800
4	0002D8	2017-08-07 21:40:00+03	23600

```
flights = pd.read_sql_query(f"""SELECT * FROM flights """, connection)
flights.head()
```

[8] ✓ 0.1s Python

	flight_id	flight_no	scheduled_departure	scheduled_arrival	departure_airport	arrival_airport	status	aircraft_code	actual_departure	actual_arri
0	1185	PG0134	2017-09-10 09:50:00+03	2017-09-10 14:55:00+03	DME	BTK	Scheduled	319		\N
1	3979	PG0052	2017-08-25 14:50:00+03	2017-08-25 17:35:00+03	VKO	HMA	Scheduled	CR2		\N
2	4739	PG0561	2017-09-05 12:30:00+03	2017-09-05 14:15:00+03	VKO	AER	Scheduled	763		\N
3	5593	PG0530	2017-09-12	2017-09-12	VKO	HMA	Scheduled	763		\N

Data Analysis - Airlines.ipynb • travel.sqlite

C: > Users > thanv > Downloads > Data Analysis - Airlines.ipynb > Objective

Generate + Code + Markdown Run All Restart Clear All Outputs Jupyter Variables Outline Python 3.13.9

```
seats = pd.read_sql_query(f"""SELECT * FROM seats """, connection)
seats.head()
```

[9] ✓ 0.0s Python

	aircraft_code	seat_no	fare_conditions
0	319	2A	Business
1	319	2C	Business
2	319	2D	Business
3	319	2F	Business
4	319	3A	Business

```
ticket_flights = pd.read_sql_query(f"""SELECT * FROM ticket_flights """, connection)
ticket_flights.head()
```

[10] ✓ 2.2s Python

	ticket_no	flight_id	fare_conditions	amount
0	0005432159776	30625	Business	42100
1	0005435212351	30625	Business	42100
2	0005435212386	30625	Business	42100
3	0005435212381	30625	Business	42100
4	0005432211370	30625	Business	42100

Data Analysis -Airlines.ipynb • travel.sqlite

C: > Users > thanv > Downloads > Data Analysis -Airlines.ipynb > M Objective

Generate + Code + Markdown Run All Restart Clear All Outputs Jupyter Variables Outline Python 3.13.9

```
tickets = pd.read_sql_query(f"""SELECT * FROM tickets """, connection)
tickets.head()
```

[11] ✓ 0.6s Python

...

	ticket_no	book_ref	passenger_id
0	0005432000987	06B046	8149 604011
1	0005432000988	06B046	8499 420203
2	0005432000989	E170C3	1011 752484
3	0005432000990	E170C3	4849 400049
4	0005432000991	F313DD	6615 976589

```
for table in table_list:
    print("\ntable: " + table)
    columns_info = connection.execute("PRAGMA table_info({})".format(table))
    for column in columns_info.fetchall():
        print(column[1:3])
```

[12] ✓ 0.0s Python

...

```
table: aircrafts_data
('aircraft_code', 'character(3)')
('model', 'jsonb')
('range', 'INTEGER')
```

Data Analysis -Airlines.ipynb • travel.sqlite

C: > Users > thanv > Downloads > Data Analysis -Airlines.ipynb > M Objective

Generate + Code + Markdown Run All Restart Clear All Outputs Jupyter Variables Outline Python 3.13.9

```
('model', 'jsonb')
('range', 'INTEGER')

table: airports_data
('airport_code', 'character(3)')
('airport_name', 'jsonb')
('city', 'jsonb')
('coordinates', 'point')
('timezone', 'TEXT')

table: boarding_passes
('ticket_no', 'character(13)')
('flight_id', 'INTEGER')
('boarding_no', 'INTEGER')
('seat_no', 'character varying(4)')

table: bookings
('book_ref', 'character(6)')
('book_date', 'timestamp with time zone')
('total_amount', 'numeric(10,2)')

table: flights
...
table: tickets
('ticket_no', 'character(13)')
('book_ref', 'character(6)')
('passenger_id', 'character varying(20)')
Output is truncated. View as a scrollable element or open in a text editor. Adjust cell output settings...
```

Generate + Code + Markdown

Data Analysis -Airlines.ipynb • travel.sqlite

C: > Users > thanv > Downloads > Data Analysis -Airlines.ipynb > Objective

Generate + Code + Markdown Run All Restart Clear All Outputs Jupyter Variables Outline Python 3.13.9

Data Cleaning

```
# checking for missing values in each column for every table
for table in table_list:
    print(f'\nMissing Values in table {table}')
    df_table = pd.read_sql_query(f"SELECT * FROM {table}", connection)
    print(df_table.isnull().sum())
```

[13] ✓ 52s Python

```
...
Missing Values in table aircrafts_data
aircraft_code    0
model            0
range           0
dtype: int64

Missing Values in table airports_data
airport_code     0
airport_name     0
city            0
coordinates     0
timezone        0
dtype: int64

Missing Values in table boarding_passes
ticket_no       0
flight_id       0
dtype: int64
```

Data Analysis -Airlines.ipynb • travel.sqlite

C: > Users > thanv > Downloads > Data Analysis -Airlines.ipynb > Objective

Generate + Code + Markdown Run All Restart Clear All Outputs Jupyter Variables Outline Python 3.13.9

```
Missing Values in table boarding_passes
ticket_no       0
flight_id       0
boarding_no     0
seat_no        0
dtype: int64

Missing Values in table bookings
book_ref        0
book_date      0
...
ticket_no      0
book_ref       0
passenger_id   0
dtype: int64
```

Output is truncated. View as a [scrollable element](#) or open in a [text editor](#). Adjust cell output [settings](#)...

Basic Analysis

How many planes have more than 100 seats?

```
pd.read_sql_query(f"SELECT aircraft_code, COUNT(*) as num_seats FROM seats
GROUP BY aircraft_code
HAVING num_seats > 100
ORDER BY num_seats DESC", connection)
```

[14]

Data Analysis - Airlines.ipynb • travel.sqlite

C: > Users > thanv > Downloads > Data Analysis -Airlines.ipynb > M+ Objective

Generate + Code + Markdown | Run All Restart Clear All Outputs | Jupyter Variables Outline ... Python 3.13.9

Basic Analysis

How many planes have more than 100 seats?

```
pd.read_sql_query(f"""SELECT aircraft_code, COUNT(*) as num_seats FROM seats
GROUP BY aircraft_code
HAVING num_seats > 100
ORDER BY num_seats DESC""", connection)
```

[14] ✓ 0.0s Python

	aircraft_code	num_seats
0	773	402
1	763	222
2	321	170
3	320	140
4	733	130
5	319	116

How the number of tickets booked and total amount earned changed with the time.

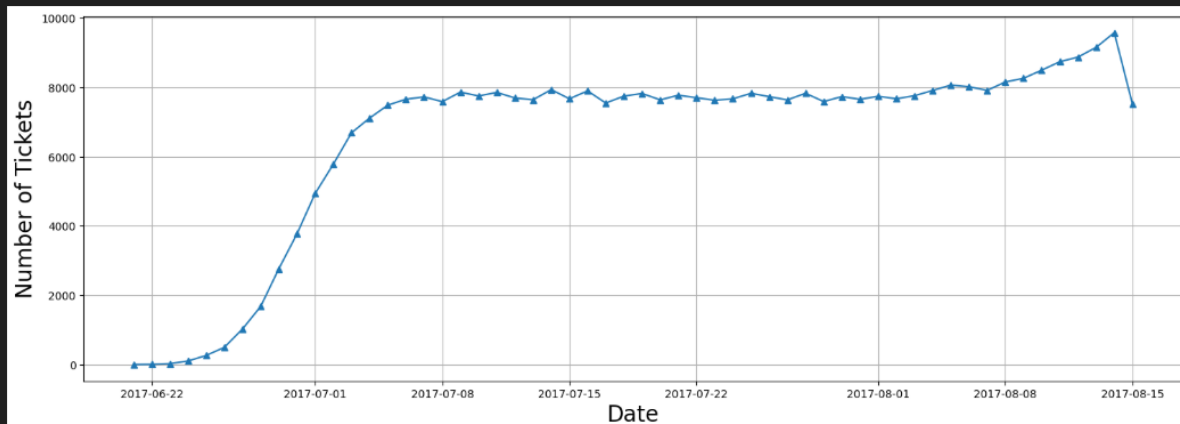
```
tickets = pd.read_sql_query(f"""SELECT *
```

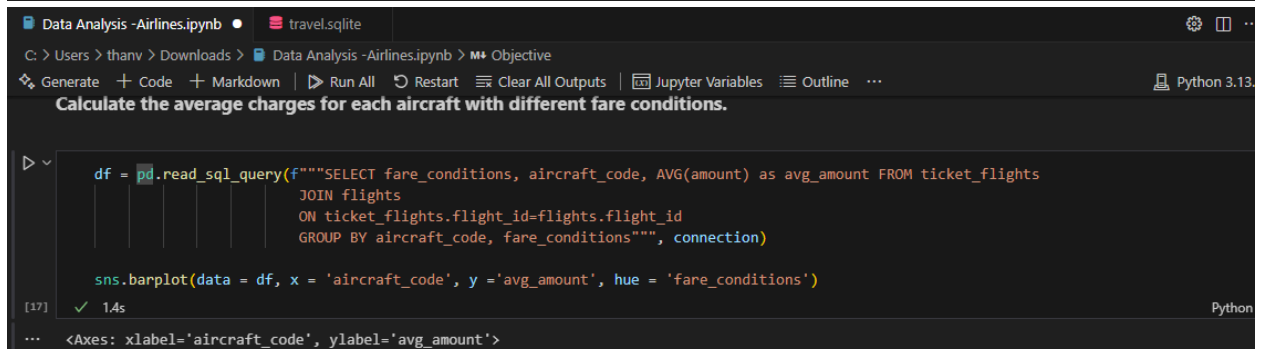
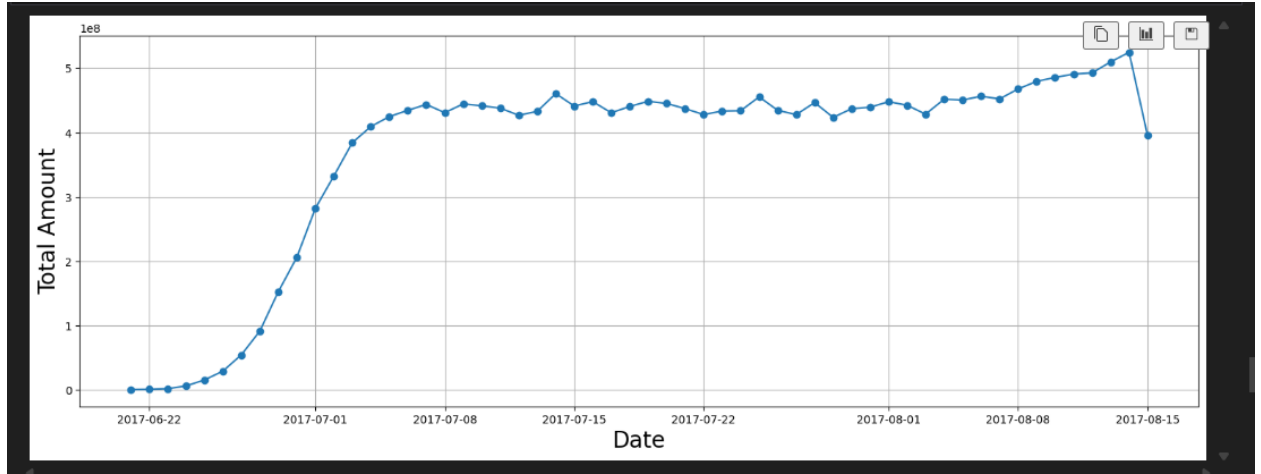
How the number of tickets booked and total amount earned changed with the time.

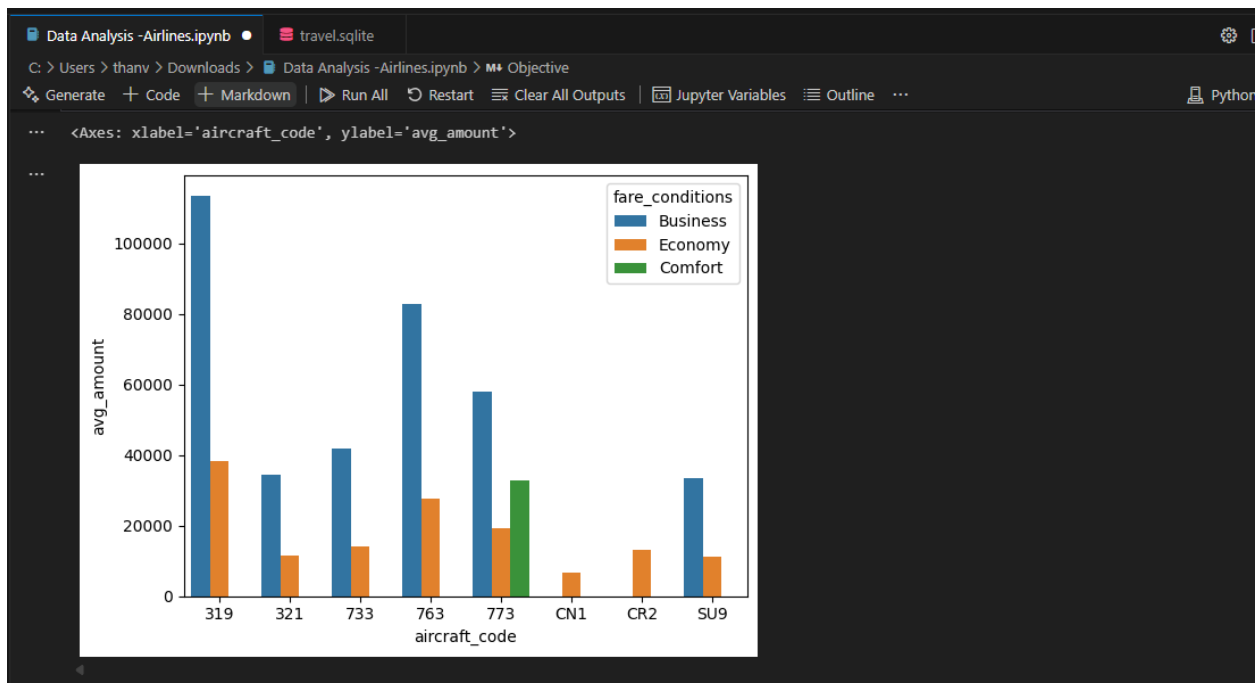
```
tickets = pd.read_sql_query(f"""SELECT *
FROM tickets
INNER JOIN bookings
ON tickets.book_ref=bookings.book_ref;""", connection)
tickets['book_date'] = pd.to_datetime(tickets['book_date'])
tickets['date'] = tickets['book_date'].dt.date
x = tickets.groupby('date')[['date']].count()
plt.figure(figsize = (18,6))
plt.plot(x.index,x['date'], marker = '^')
plt.xlabel('Date', fontsize = 20)
plt.ylabel('Number of Tickets', fontsize = 20)
plt.grid('b')
plt.show()
```

[1] ✓ 6.1s Python

```
plt.grid('b')
plt.show()
✓ 6.1s Python
```







Data Analysis -Airlines.ipynb • travel.sqlite

C: > Users > thanv > Downloads > Data Analysis -Airlines.ipynb > M Objective

Generate + Code + Markdown | Run All Restart Clear All Outputs | Jupyter Variables Outline ... Python 3.13.9

Analyzing occupancy rate

For each aircraft, calculate the total revenue per year and the average revenue per ticket.

```
pd.set_option('display.float_format', str)
```

[18] ✓ 0.0s Python

```
pd.read_sql_query(f"""SELECT aircraft_code, total_revenue, ticket_count, total_revenue/ticket_count as avg_revenue_per_ticket
FROM
(SELECT aircraft_code, COUNT(*) as ticket_count, SUM(amount) as total_revenue FROM ticket_flights
JOIN flights
ON ticket_flights.flight_id=flights.flight_id
GROUP BY aircraft_code)""", connection)
```

[19] ✓ 0.7s Python

	aircraft_code	total_revenue	ticket_count	avg_revenue_per_ticket
0	319	2706163100	52853	51201
1	321	1638164100	107129	15291
2	733	1426552100	86102	16568
3	763	4371277100	124774	35033
4	773	3431205500	144376	23765
5	CN1	96373800	14672	6568

Data Analysis -Airlines.ipynb

travel.sqlite

C: > Users > thanv > Downloads > Data Analysis -Airlines.ipynb > M+ Objective

Generate + Code + Markdown Run All Restart Clear All Outputs Jupyter Variables Outline ... Python 3.13.9

4	773	3431205500	144376	23765
5	CN1	96373800	14672	6568
6	CR2	1982760500	150122	13207
7	SU9	5114484700	365698	13985

Calculate the average occupancy per aircraft.

```
occupancy_rate = pd.read_sql_query(f"""SELECT a.aircraft_code, AVG(a.seats_count) as booked_seats, b.num_seats,
AVG(a.seats_count)/b.num_seats as occupancy_rate
FROM (
    SELECT aircraft_code, flights.flight_id, COUNT(*) as seats_count
    FROM boarding_passes
    INNER JOIN flights
    ON boarding_passes.flight_id=flights.flight_id
    GROUP BY aircraft_code, flights.flight_id
) as a INNER JOIN
(
    SELECT aircraft_code, COUNT(*) as num_seats FROM seats
    GROUP BY aircraft_code
) as b
ON a.aircraft_code = b.aircraft_code
GROUP BY a.aircraft_code""", connection)

occupancy_rate
```

[20] ✓ 0.5s Python

Data Analysis -Airlines.ipynb

travel.sqlite

C: > Users > thanv > Downloads > Data Analysis -Airlines.ipynb > M+ Objective

Generate + Code + Markdown Run All Restart Clear All Outputs Jupyter Variables Outline ... Python 3.13.9

[20] ✓ 0.5s Python

	aircraft_code	booked_seats	num_seats	occupancy_rate
0	319	53.58318098720292	116	0.46192397402761143
1	321	88.80923076923077	170	0.5224072398190045
2	733	80.25546218487395	130	0.617349709114415
3	763	113.93729372937294	222	0.5132310528350132
4	773	264.9258064516129	402	0.659019419033863
5	CN1	6.004431314623338	12	0.5003692762186115
6	CR2	21.48284690220174	50	0.42965693804403476
7	SU9	56.81211267605634	97	0.5856918832583128

Calculate by how much the total annual turnover could increase by giving all aircraft a 10% higher occupancy rate.

```
occupancy_rate['Inc occupancy rate'] = occupancy_rate['occupancy_rate'] + occupancy_rate['occupancy_rate']*0.1
occupancy_rate
```

[21] ✓ 0.0s Python

	aircraft_code	booked_seats	num_seats	occupancy_rate	Inc occupancy rate
0	319	53.58318098720292	116	0.46192397402761143	0.5081163714303726
1	321	88.80923076923077	170	0.5224072398190045	0.574647963800905
2	733	80.25546218487395	130	0.617349709114415	0.6790846800258565
3	763	113.93729372937294	222	0.5132310528350132	0.5645541581185146

Data Analysis - Airlines.ipynb • travel.sqlite

C: > Users > thanv > Downloads > Data Analysis - Airlines.ipynb > Objective

Generate + Code + Markdown Run All Restart Clear All Outputs Jupyter Variables Outline Python 3.13.9

4	773	264.9258064516129	402	0.659019419033863	0.7249213609372492	
5	CN1	6.004431314623338	12	0.5003692762186115	0.5504062038404727	
6	CR2	21.48284690220174	50	0.42965693804403476	0.4726226318484382	
7	SU9	56.81211267605634	97	0.5856918832583128	0.644261071584144	

```
total_revenue = pd.read_sql_query("""SELECT aircraft_code, SUM(amount) as total_revenue FROM ticket_flights
JOIN flights
ON ticket_flights.flight_id=flights.flight_id
GROUP BY aircraft_code""", connection)

occupancy_rate['Inc Total Annual Turnover'] = (total_revenue['total_revenue']/occupancy_rate['occupancy_rate'])*occupancy_rate['Inc occupancy_rate']
```

[22] ✓ 0.7s Python

	aircraft_code	booked_seats	num_seats	occupancy_rate	Inc occupancy rate	Inc Total Annual Turnover
0	319	53.58318098720292	116	0.46192397402761143	0.5081163714303726	2976779410.0
1	321	88.80923076923077	170	0.5224072398190045	0.574647963800905	1801980510.0
2	733	80.25546218487395	130	0.617349709114415	0.6790846800258565	1569207310.0000002
3	763	113.93729372937294	222	0.5132310528350132	0.5645541581185146	4808404810.0
4	773	264.9258064516129	402	0.659019419033863	0.7249213609372492	3774326050.0
5	CN1	6.004431314623338	12	0.5003692762186115	0.5504062038404727	106011180.00000001
6	CR2	21.48284690220174	50	0.42965693804403476	0.4726226318484382	2181036550.0
7	SU9	56.81211267605634	97	0.5856918832583128	0.644261071584144	5625933169.999999

Data Analysis - Airlines.ipynb • travel.sqlite

C: > Users > thanv > Downloads > Data Analysis - Airlines.ipynb > Objective

Generate + Code + Markdown Run All Restart Clear All Outputs Jupyter Variables Outline Python 3.13.9

```
total_revenue = pd.read_sql_query("""SELECT aircraft_code, SUM(amount) as total_revenue FROM ticket_flights
JOIN flights
ON ticket_flights.flight_id=flights.flight_id
GROUP BY aircraft_code""", connection)

occupancy_rate['Inc Total Annual Turnover'] = (total_revenue['total_revenue']/occupancy_rate['occupancy_rate'])*occupancy_rate['Inc occupancy_rate']
```

[22] ✓ 0.7s Python

	aircraft_code	booked_seats	num_seats	occupancy_rate	Inc occupancy rate	Inc Total Annual Turnover
0	319	53.58318098720292	116	0.46192397402761143	0.5081163714303726	2976779410.0
1	321	88.80923076923077	170	0.5224072398190045	0.574647963800905	1801980510.0
2	733	80.25546218487395	130	0.617349709114415	0.6790846800258565	1569207310.0000002
3	763	113.93729372937294	222	0.5132310528350132	0.5645541581185146	4808404810.0
4	773	264.9258064516129	402	0.659019419033863	0.7249213609372492	3774326050.0
5	CN1	6.004431314623338	12	0.5003692762186115	0.5504062038404727	106011180.00000001
6	CR2	21.48284690220174	50	0.42965693804403476	0.4726226318484382	2181036550.0
7	SU9	56.81211267605634	97	0.5856918832583128	0.644261071584144	5625933169.999999